



Convergence and correctness of belief propagation for weighted min–max flow

Guowei Dai^a, Longkun Guo^b, Gregory Gutin^c, Xiaoyan Zhang^{a,*},
Zan-Bo Zhang^d

^a School of Mathematical Science & Institute of Mathematics, Nanjing Normal University, Nanjing, China

^b Department of Computer Science, Qilu University of Technology, Jinan, China

^c Department of Computer Science, Royal Holloway University of London, Egham, UK

^d School of Statistics & Mathematics, and Institute of Artificial Intelligence & Deep Learning, Guangdong University of Finance & Economics, Guangzhou, China

ARTICLE INFO

Article history:

Received 9 March 2021

Received in revised form 21 November 2021

Accepted 13 December 2021

Available online 13 January 2022

Keywords:

Belief propagation

Min–max BP algorithm

Message-passing algorithm

Min–max flow

ABSTRACT

In this paper, we investigate the performance of message-passing algorithms for the weighted min–max flow ($\mathcal{WMMF}$) problem which was introduced by Ichimori et al. (1980). $\mathcal{WMMF}$ was well studied in combinational optimization, as it provides important applications in time transportation problem and the storage management problem. We develop a message-passing algorithm called min–max belief propagation (BP) for determining the optimal solution of $\mathcal{WMMF}$. As the main result of this paper, we prove that for a digraph of size n , BP converges to the optimal solution within $O(n^3)$ time after $O(n)$ iterations if the optimal solution of the underlying min–max flow problem instance is unique. To the best of our knowledge, the fastest polynomial time algorithm for $\mathcal{WMMF}$ runs in essentially $O(n^6)$ time among the known algorithms, where n is the number of vertices. On the other hand, it is one of a very few instances where BP is proved correct with fully-polynomial running time.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

As an algorithmic framework, message passing is extremely powerful and has been widely used on various graphical models (GMs). Belief propagation (BP), proposed by Pearl in 1988 [17], is a message-passing heuristic algorithm and has wide applications in the context of variety of disciplines including satisfiability in discrete optimization [1,16], error correcting code in information theory [15,18], and data clustering in machine learning [9]. The great popularity of BP can be attributed to two main reasons. Firstly, it is easy to implement due to its simple and message-passing nature. Secondly, it performs well in many practical applications. The wide scope of application, simplicity, and experimental success of BP has gained a lot of attention recently [16,18,23].

BP is known to converge to the correct solution on GMs with no cycles [17]. When the underlying graph is a tree, the BP algorithm essentially performs the recursion of dynamic programming (DP) leading to a correct solution. Specifically, BP provides a natural parallel iterative version of the DP in which message passing occurs between the variable nodes along edges of the graphical model. Surprisingly, even for GMs with cycles, the BP heuristic performs well in many cases [22],

* Corresponding author.

E-mail addresses: guoweidai@njnu.edu.cn (G. Dai), longkun.guo@qlu.edu.cn (L. Guo), gutin@cs.rhul.ac.uk (G. Gutin), zhangxiaoyan@njnu.edu.cn (X. Zhang), zanbozhang@gdufe.edu.cn (Z.-B. Zhang).

some of which require rigorous analysis of optimality and convergence [3,7,11,19,20]. Unfortunately, the correctness and convergence properties of BP for general combinatorial optimization problems are still open.

As a major breakthrough, Bayati et al. [3] and Cheng et al. [6] were the first to simplify the BP algorithm independently to obtain essentially the same algorithms for the maximum weight matching (MWM) in a bipartite graph. They established the correctness and convergence of BP algorithm for MWM in pseudo-polynomial time. Sanghavi et al. [19] as well as Bayati et al. [2] generalized the result to the minimum cost b -matching problem on arbitrary graphs and established that BP algorithm converges to the optimal solution, in pseudo-polynomial time, as long as the corresponding linear programming relaxation has no fractional solutions. Furthermore, MWM can be viewed as a special case of the minimum cost flow (MCF) problem. Recently, Gamarnik et al. [11] studied the performance of BP algorithm for finding the optimal solution of MCF and proved that BP algorithm converges to the optimal solution in pseudo-polynomial time, provided that the optimal solution is unique. Brunsch et al. [4] studied BP algorithm in the framework of smoothed analysis and proved that with high probability the number of iterations needed to compute maximum-weight matchings and min-cost flows is bounded by a polynomial if the weights or costs of the edges are randomly perturbed.

The running time of BP algorithm converging to the optimum is actually pseudo-polynomial for some combinatorial optimization problems, although most of those optimization problems (e.g., MWM and MCF) have other fully polynomial time algorithms. Gamarnik et al. [11] also presented a simple modification of BP algorithm to obtain a fully polynomial time randomized approximation scheme for MCF. However, as they said themselves in [11], the ‘near optimal’ solution is ‘rather fuzzy’. In order to identify the class of optimization problems solvable in fully polynomial time using the BP algorithm, we study the weighted min-max flow ($\mathcal{WMMF}$) problem and develop a min-max BP algorithm for determining the optimal solution of $\mathcal{WMMF}$. As a variant of the maximum flow problems, $\mathcal{WMMF}$ was introduced by Ichimori et al. [14] and was well studied in combinatorial optimization [8,10,12], as it provides important applications in time transportation problem [5] and the storage management problem [21].

In this paper, we will investigate the convergence and correctness of the min-max BP algorithm for finding the optimal solution of $\mathcal{WMMF}$ on arbitrary digraphs. As the main result, we establish that our algorithm converges to the optimal solution of $\mathcal{WMMF}$ after at most $n/2$ iterations where n represents the number of vertices, provided that the optimal solution is unique. From the description of min-max BP algorithm, it is seen that each of the messages can be computed in $O(n^2)$ time. Then due to the distributed nature of BP algorithm, the computational cost of the algorithm is $O(n^3)$ in $O(n)$ iterations. As a result, the min-max BP algorithm we developed is a fully polynomial time algorithm. On the one hand, our algorithm is one of a very few instances where BP is proved correct with fully-polynomial running time. On the other hand, to the best of our knowledge, the fastest polynomial time algorithm for $\mathcal{WMMF}$ runs in essentially $O(n^6)$ time [13] among the known algorithms, where n is the number of vertices. According to our theoretical analysis, it may explain why BP can perform well for most of combinatorial optimization problems and run fast in practice.

The rest of the paper is organized as follows. In Section 2, we introduce the weighted min-max flow problem ($\mathcal{WMMF}$). In Section 3, we describe the min-max BP algorithm for $\mathcal{WMMF}$, and state our main result. Proofs of correctness and convergence for our algorithm are given in Section 4. Finally, Section 5 presents the conclusions and directions for future research.

2. Preliminaries

2.1. Definitions and problem statement

Given a weighted digraph (i.e., *network*) $G = (V, E)$ where V, E denote the set of vertices and arcs, respectively with $|V| = n, |E| = m$. To each arcs $e \in E$, assign a nonnegative weight w_e and a positive capacity c_e . For given source $s \in V$ and sink $t \in V$, denoted by f^* the total value of given flow from s to t . For any vertex $i \in V$, let E_i^- be the set of arcs entering i , that is, $E_i^- = \{ji : ji \in E\}$. Similarly, $E_i^+ = \{ij : ij \in E\}$ denotes the set of arcs leaving i . Let $E_i = E_i^- \cup E_i^+$. We assume for simplicity that $|E_s^-| = |E_t^+| = 0$. The weighted min-max flow ($\mathcal{WMMF}$) problem aims to minimize the maximum value of weighted arc-flow (multiplied by arc-weight). Then, the $\mathcal{WMMF}$ on G can be formulated as the following linear program:

$$\min \quad \max_{e \in E} w_e x_e \quad (I)$$

$$\text{s.t.} \quad \sum_{e \in E_i^+} x_e - \sum_{e \in E_i^-} x_e = f_i = \begin{cases} f^*, & i = s; \\ -f^*, & i = t; \\ 0, & i \in V \setminus \{s, t\}, \end{cases} \quad (1)$$

$$0 \leq x_e \leq c_e, \quad \forall e \in E, \quad (2)$$

where the variables x_e represent the flow value assigned to each arc $e \in E$. The constraints (1) state that the difference of out-flow and in-flow at each vertex $i \in V$ equals the vertex demand f_i . The constraints (2) state that flow value on each arc $e \in E$ is nonnegative and at most its capacity c_e . Note that to enable the instance of network flow to be feasible, we assume w.l.o.g. that $|E_i| \geq 2$ for each $i \in V$, or else $|E_i| = 1$, in which case the flow on $e \in E_i$ is determined by f_i .

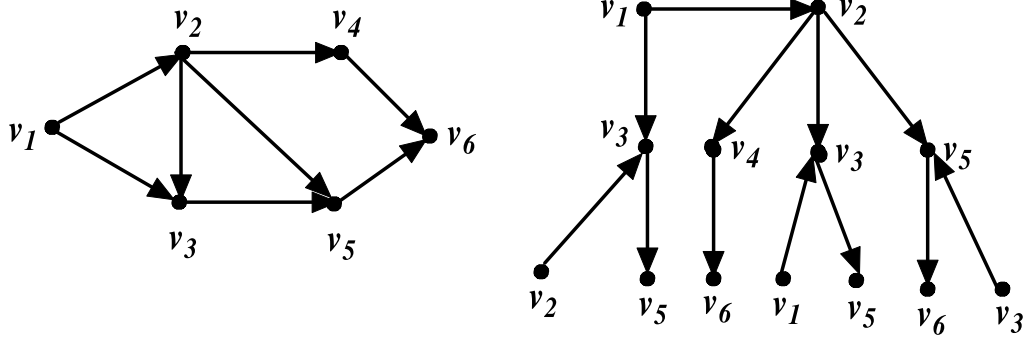


Fig. 1. An example of a 2-level computation tree $T_{v_1 v_2}^2$ with root $v_1 v_2$.

2.2. Factorized optimization problem

Consider the optimization problem ($\mathcal{P}$) as follows:

$$\begin{aligned} \min \quad & \sum_{i \in V} \phi_i(x_i) + \sum_{D \in \mathcal{D}} \psi_D(x_D) \\ \text{s.t.} \quad & x_i \in \mathbb{R}, \quad \forall i \in V, \end{aligned}$$

where V is a finite set of variables and $\mathcal{D}$ is a finite collection of subsets of V representing constraints. Here $\phi_i : \mathbb{R} \rightarrow \mathbb{R} \cup \{\infty\}$ and $\psi_D : \mathbb{R}^{|D|} \rightarrow \mathbb{R} \cup \{\infty\}$, $\forall D \in \mathcal{D}$ are extended real-valued functions, where each ϕ_i is called a variable function and each ψ_D is called a factor function. We also call the optimization problem ($\mathcal{P}$) a *factorized optimization problem*.

It is well-known that BP algorithms are always viewed as heuristic algorithms for factorized optimization problems and operate by passing messages iteratively with variables and factors. Next, we translate (I) into a factorized optimization problem. Let $x_{E_i} = \{x_e : e \in E_i\}$ for each $i \in V$. Define variable and factor functions ϕ, ψ for each $e \in E, i \in V$, respectively as follows: $\phi_e(x_e) = w_e x_e$ and

$$\psi_i(x_{E_i}) = \begin{cases} 0 & \text{if } \sum_{e \in E_i^+} x_e - \sum_{e \in E_i^-} x_e = f_i, \\ +\infty & \text{otherwise.} \end{cases}$$

Then, we can formulate $\mathcal{WMMF}$ as an unconstrained optimization problem as follows.

$$\begin{aligned} \min \quad & \max_{e \in E, i \in V} \{ \phi_e(x_e), \psi_i(x_{E_i}) \} \\ \text{s.t.} \quad & 0 \leq x_e \leq c_e, \quad \forall e \in E. \end{aligned}$$

3. BP algorithm for $\mathcal{WMMF}$

3.1. Computation tree

Here we introduce the concept of *rooted tree* and *computation tree*. A connected acyclic graph (i.e., contains no cycles) is called a *tree*. For any nontrivial tree, it must contain a vertex which has exactly one neighbor. Such a vertex in a tree is also called a *leaf* of the tree. Throughout of the paper, we define a rooted tree T_a as a tree T with a specified arc a , called the *root* of T . It should be noted that the definition of the root of a tree sometimes refers to a specified vertex, in contrast to a root as the root. In a tree, any two vertices are connected by exactly one path. We denote the unique path connecting vertices i and j in a tree T by iTj . For a rooted tree T_a with root a , the *level* of a vertex i in T_a is the length of the path aTi , and each vertex on the path aTi is called an *ancestor* of i . For two adjacent vertices i, j in T , if i is an ancestor of j , then i is also called a *parent* of j , and j is a *child* of i .

We use T_a^N to denote the N -level computation tree associated with arc a as the root. An example of computation tree is given in Fig. 1. Denote the set of vertices and arcs in T_a^N by $V(T_a^N)$ and $E(T_a^N)$, respectively. Each vertex or arc of T_a^N is a duplicate of some vertex or arc of the original graph G . Define the mapping $\gamma_a^N : V(T_a^N) \rightarrow V$ such that if $i' \in V(T_a^N)$ is a duplicate of $i \in V(G)$, then $\gamma_a^N(i') = i$. Denote by $L(T_a^N)$ the set of leaves of T_a^N . For any $i' \in V(T_a^N)$, denote by $P(i')$ the parent of i' in T_a^N . It is essentially the breadth-first search tree of G (with repetition of vertices allowed) starting from a up to depth N . In detail, we inductively define T_a^N as the following rules.

- Let $uv \in E(G)$. Then the computation tree T_a^0 consists of two vertices u' , v' and an arc $u'v'$, such that $\gamma_a^0(u') = u$ and $\gamma_a^0(v') = v$. The arc $a = u'v'$ is considered the “root” of T_a^0 , and vertices u' , v' are considered to be at 0-level of T_a^0 .
- Inductively, suppose that we defined a tree T_a^{N-1} , such that for any $i', j' \in V(T_a^{N-1})$, $i'j' \in E(T_a^{N-1})$ if and only if $\gamma_a^{N-1}(i')\gamma_a^{N-1}(j') \in E(G)$. The computation tree T_a^N contains T_a^{N-1} as a subtree, which can be obtained by adding vertices to $V(T_a^{N-1})$ and arcs to $E(T_a^{N-1})$ as follows. For each leaf vertex $i' \in L(T_a^{N-1})$, add node j' to expand $V(T_a^{N-1})$ and add arc $i'j'$ or $j'i'$ to expand $E(T_a^{N-1})$ if there is a vertex $j \in V(G)$ such that $ij \in E(G)$ or $ji \in E(G)$ with $\gamma_a^{N-1}(i') = i$, and $\gamma_a^{N-1}(P(i')) \neq j$. In this case, define $P(j') = i'$, the map $\gamma_a^N(j') = j$, and level of j' as N . Indeed, γ_a^N is identical to γ_a^{N-1} for vertices in $V(T_a^{N-1}) \subseteq V(T_a^N)$.
- For any $e = ij \in E(G)$, the arc from i' to j' in T_a^N is also denoted by e for simplicity and is assigned the same weight w_e as that in G , where $\gamma_a^N(i') = i$ and $\gamma_a^N(j') = j$.

In what follows, we shall drop reference to a , t in notation of γ_a^N when clear from context and abuse notation by denoting $\gamma(i'j') = \gamma(i')\gamma(j')$.

Note that the computation tree is locally equivalent to the original graph, which means one can view the iterative process of BP algorithm as sending the messages along the way from leaf vertices to the root in the computation tree. All the vertices on computation tree will send messages to their parents at each iteration, and the direction of message-passing is independent of the direction of those edges.

Given a root a , let $V^o(T_a^N) \subset V(T_a^N)$ be the set of all the non-leaf vertices of T_a^N . Recall that for any vertex $k \in V(T_a^N)$, $E_k^+(T_a^N)$ and $E_k^-(T_a^N)$ denote the set of arcs leaving k and entering k in T_a^N , respectively. Then we consider the problem formulated as $\mathcal{WMMF}_a^N$:

$$\min \max_{e \in E(T_a^N)} w_e y_e \quad (II)$$

$$\text{s.t.} \quad \sum_{e \in E_k^+(T_a^N)} y_e - \sum_{e \in E_k^-(T_a^N)} y_e = f_{\gamma(k)}, \quad \forall k \in V^o(T_a^N); \quad (3)$$

$$0 \leq y_e \leq c_e, \quad \forall e \in E(T_a^N), \quad (4)$$

where the variables y_e represent flow value assigned to each arc $e \in E(T_a^N)$. Roughly speaking, $\mathcal{WMMF}_a^N$ is a $\mathcal{WMMF}$ on the computation tree T_a^N essentially: there is a network flow capacity constraint for every arc $e \in E(T_a^N)$ and a flow balance constraint for every vertex, except for the leaf vertices.

3.2. Algorithm and results

For each arc $e = ij$ on the computation tree T_a^N , by definition, T_a^N has two components which are connected via the arc e . Let $T_{e \rightarrow j}^N$ be the component containing i , which can be viewed as a subtree of T_a^N . Let $T_{i \rightarrow e}^N := T_{e \rightarrow j}^N \cup \{e\}$. Define a message function $m_{e \rightarrow j}(x_e)$ on the subtree $T_{e \rightarrow j}^N$. Let the function $m_{e \rightarrow j}(x_e)$ return the maximum value of arc-flow (multiplied by arc-weight) of the $\mathcal{WMMF}$ on the subtree $T_{e \rightarrow j}^N$. Similarly, define the message function $m_{i \rightarrow e}(x_e)$ which returns the maximum value of arc-flow (multiplied by arc-weight) of the $\mathcal{WMMF}$ on the subtree $T_{i \rightarrow e}^N$. Due to the nature of tree structure, these two message functions can be recursively defined as follows: for any arc $e = ij$,

$$m_{e \rightarrow j}(x_e) = \max \left\{ \phi_e(x_e), m_{i \rightarrow e}(x_e) \right\}, \quad (5)$$

$$m_{i \rightarrow e}(x_e) = \min_{x_{E_i \setminus e}} \left\{ \max \left\{ \psi_i(x_{E_i}), \max_{e' \in E_i \setminus e} m_{e' \rightarrow i}(x_{e'}) \right\} \right\}. \quad (6)$$

Using (5)–(6), starting from leaves, the message functions $m_{e \rightarrow j}(x_e)$ and $m_{i \rightarrow e}(x_e)$ can be computed for all $e \in E$, $i \in V$. Then, the update messages for each vertex and arc is as follows:

$$m_{e \rightarrow j}^t(x_e) = \max \left\{ \phi_e(x_e), m_{i \rightarrow e}^{t-1}(x_e) \right\},$$

$$m_{i \rightarrow e}^t(x_e) = \min_{x_{E_i \setminus e}} \left\{ \max \left\{ \psi_i(x_{E_i}), \max_{e' \in E_i \setminus e} m_{e' \rightarrow i}^t(x_{e'}) \right\} \right\}.$$

Finally, combine the messages $m_{a \rightarrow u}(x_a)$ and $m_{a \rightarrow v}(x_a)$ at the root arc $a = uv$, we can derive the estimation of *belief* at the end of iteration N on the computation tree T_a^N as

$$b_a^N(x_a) = \max \left\{ m_{a \rightarrow u}^N(x_a), m_{a \rightarrow v}^N(x_a) \right\}.$$

The parallel algorithm called min-max BP for solving $\mathcal{WMMF}$ is described in detail as Algorithm 1.

Next, we will state our result, the proof of which is presented in Section 4.

Algorithm 1 min-max BP for $\mathcal{WMMF}$

1: Initialize $t = 0$, message $m_{i \rightarrow e}^0(x_e) = w_e x_e$ for each $e = ij \in E$.

2: **for** $t = 1, 2, \dots, N$ **do**

3: For each $e = ij \in E$ update messages as follows:

$$m_{e \rightarrow j}^t(x_e) = \max \left\{ \phi_e(x_e), m_{i \rightarrow e}^{t-1}(x_e) \right\},$$

$$m_{i \rightarrow e}^t(x_e) = \min_{x_{E_i \setminus e}} \left\{ \max \left\{ \psi_i(x_{E_i}), \max_{e' \in E_i \setminus e} m_{e' \rightarrow i}^t(x_{e'}) \right\} \right\}.$$

4: $t := t + 1$

5: **end for**

6: For each $e = ij \in E$, set the belief function as $b_e^N(x_e) = \max \left\{ m_{e \rightarrow i}^N(x_e), m_{e \rightarrow j}^N(x_e) \right\}$.

7: Calculate the belief estimate by finding $\hat{x}_e^N \in \arg \min_{0 \leq x_e \leq c_e} b_e^N(x_e)$ for each $e \in E$.

8: Return $\hat{\mathbf{x}}^N = \{\hat{x}_e^N : e \in E\}$ as an estimation of the optimal solution.

Theorem 1. For any digraph G of order n , if the problem $\mathcal{WMMF}$ on G has a unique optimal solution x^* , then Algorithm 1 converges to x^* after $\frac{n}{2}$ iterations, i.e., $\hat{\mathbf{x}}^N = x^*$ after $N \geq \frac{n}{2}$ iterations.

From the description of min-max BP algorithm, it is seen that each of the messages can be computed in $O(n^2)$ time. Then due to the distributed nature of BP algorithm, the total computational time of our algorithm is $O(n^3)$ in $\frac{n}{2}$ iterations. As a corollary of Theorem 1, the min-max BP algorithm we developed is a fully polynomial time algorithm.

Corollary 1. For any digraph G of order n , if the problem $\mathcal{WMMF}$ on G has a unique optimal solution x^* , then min-max BP algorithm converges to x^* in $O(n^3)$ time.

4. Proof of correctness and convergence

In this section, we will establish the convergence of BP to the optimal solution of the $\mathcal{WMMF}$ under the assumption of the uniqueness of the optimal solution, namely we shall prove Theorem 1. Note that our strategy is somewhat similar to that of [11], but the technical details are quite different.

Lemma 2. Let $\hat{x}_a^N$ be the value of the output of the BP algorithm at the end of iteration N on arc $a \in E$. Then there exists an optimal solution y^* of $\mathcal{WMMF}_a^N$ such that $y_a^* = \hat{x}_a^N$ where a is the root of T_a^N .

Proof. Let $a = ij$ be the root of T_a^N . By definition, T_a^N has two components connected by the arc a . Denote the component containing i by C and $T_{a \rightarrow j}^N$ denotes C with arc a (indeed $T_{a \rightarrow j}^N$ is a tree). Let $V^0(T_{a \rightarrow j}^N)$ be the set of all the vertices which are not on the N -level of $T_{a \rightarrow j}^N$. Define $\mathcal{WMMF}_{a \rightarrow j}^N(z)$ as follows.

$$\begin{aligned} \min \quad & \max_{e \in E(T_{a \rightarrow j}^N)} w_e y_e & (\mathcal{WMMF}_{a \rightarrow j}^N(z)) \\ \text{s.t.} \quad & \sum_{e \in E_k^+(T_{a \rightarrow j}^N)} y_e - \sum_{e \in E_k^-(T_{a \rightarrow j}^N)} y_e = f_{\gamma(k)}, & \forall k \in V^0(T_{a \rightarrow j}^N) \\ & y_a = z \\ & 0 \leq y_e \leq c_e, & \forall e \in E(T_{a \rightarrow j}^N). \end{aligned}$$

Now, we show that under the BP algorithm the value of $m_{a \rightarrow \gamma(j)}^N(z)$ is the same as the weight of the optimal assignment for $\mathcal{WMMF}_{a \rightarrow j}^N(z)$. This can be established inductively. When $N = 1$, the statement is easy to be checked. For $N > 1$ and each $b \in E_i \setminus a$ with $b = pi$ (or ip), let $T_{b \rightarrow i}^{N-1}$ be the subtree of $T_{a \rightarrow j}^N$ that includes b and does not include i . Consider the subproblem $\mathcal{WMMF}_{b \rightarrow i}^{N-1}(z)$ as follows.

$$\begin{aligned} \min \quad & \max_{e \in E(T_{b \rightarrow i}^{N-1})} w_e y_e & (\mathcal{WMMF}_{b \rightarrow i}^{N-1}(z)) \\ \text{s.t.} \quad & \sum_{e \in E_k^+(T_{b \rightarrow i}^{N-1})} y_e - \sum_{e \in E_k^-(T_{b \rightarrow i}^{N-1})} y_e = f_{\gamma(k)}, & \forall k \in V^0(T_{b \rightarrow i}^{N-1}) \\ & y_b = z \\ & 0 \leq y_e \leq c_e, & \forall e \in E(T_{b \rightarrow i}^{N-1}). \end{aligned}$$

By induction hypothesis, it is easy to see that the value of $m_{b \rightarrow \gamma(i)}^{N-1}(z)$ equals the weight of the solution of $\mathcal{WMF}_{b \rightarrow i}^{N-1}(z)$. Given this hypothesis and the relation of sub-tree $T_{b \rightarrow i}^{N-1}$ for all $b \in E_i \setminus a$ with $T_{a \rightarrow j}^N$, it follows that the problem $\mathcal{WMF}_{a \rightarrow j}^N(z)$ is equivalent to

$$\begin{aligned} \min \quad & \max \left\{ w_a z, \max_{b \in E_i \setminus a} m_{b \rightarrow \gamma(i)}^{N-1}(y_b) \right\} \\ \text{s.t.} \quad & \sum_{e \in E_i^+(T_{a \rightarrow j}^N)} y_e - \sum_{e \in E_i^-(T_{a \rightarrow j}^N)} y_e = f_{\gamma(i)}, \\ & y_a = z \\ & 0 \leq y_b \leq c_b, \quad \forall b \in E_i \setminus a. \end{aligned}$$

This is exactly the same as the relation between $m_{a \rightarrow \gamma(j)}^N(z)$ and message function $m_{b \rightarrow \gamma(i)}^{N-1}(\cdot)$ for $b \in E_i \setminus a$ as

$$m_{a \rightarrow j}^N(z) = \max \left\{ w_a z, \max_{b \in E_i \setminus a} m_{b \rightarrow \gamma(i)}^{N-1}(y_b) \right\}.$$

That is, $m_{a \rightarrow \gamma(j)}^N(z)$ is exactly the same as the weight of optimal assignment of $\mathcal{WMF}_{a \rightarrow j}^N(z)$. Using this equivalence, we will complete the proof of [Lemma 2](#).

For given $a = ij$ with $0 \leq z \leq c_a$, the problem $\mathcal{WMF}_a^N(z)$ is equivalent to

$$\begin{aligned} \min \quad & \max \left\{ \max_{e \in E(T_{a \rightarrow i}^N)} w_e y_e, \max_{e \in E(T_{a \rightarrow j}^N)} w_e y_e \right\} \\ \text{s.t.} \quad & \sum_{e \in E_k^+(T_a^N)} y_e - \sum_{e \in E_k^-(T_a^N)} y_e = f_{\gamma(k)}, \quad \forall k \in V^o(T_a^N) \cap (V^o(T_{a \rightarrow k}^N) \cup V^o(T_{a \rightarrow j}^N)) \\ & 0 \leq y_e \leq c_e, \quad \forall e \in E(T_{a \rightarrow i}^N) \cup E(T_{a \rightarrow j}^N). \end{aligned}$$

That is, the maximum value of weighted arc-flow of the optimal solution of $\mathcal{WMF}_a^N(z)$ equals $\max \left\{ m_{a \rightarrow \gamma(i)}^N(z), m_{a \rightarrow \gamma(j)}^N(z) \right\}$, for any $0 \leq z \leq c_a$. Now the claim of [Lemma 2](#) follows immediately. $\square$

It is clear that [Lemma 2](#) establishes the relation between BP algorithm and computation tree T_a^N . Next, we shall show the correctness and convergence of min-max BP algorithm for $\mathcal{WMF}$ as follows.

Proof of Theorem 1. To the contrary, we suppose that there is an arc $e_0 \in E$ such that $\hat{x}_{e_0}^N \neq x_{e_0}^*$ where $N > \frac{n}{2}$. We assume w.l.o.g. $\hat{x}_{e_0}^N > x_{e_0}^*$. Then, by [Lemma 2](#), there is an optimal solution y^* of $\mathcal{WMF}_{e_0}^N$ such that $y_{e_0}^* > x_{e_0}^*$.

Let $e_0 = uv$ be the root of the computation tree $T_{e_0}^N$ as above. For any vertex $i \in V^o(T_{e_0}^N)$, denote by $E_i^+(T_{e_0}^N)$ and $E_i^-(T_{e_0}^N)$ the set of arcs leaving and entering i in $T_{e_0}^N$, respectively. Since x^* and y^* are the feasible solutions of $\mathcal{WMF}$ and $\mathcal{WMF}_{e_0}^N$, respectively,

$$f_{\gamma(u)} = x_{e_0}^* + \sum_{e \in E_i^+(T_{e_0}^N) \setminus e_0} x_e^* - \sum_{e \in E_i^-(T_{e_0}^N)} x_e^*, \quad (7)$$

$$f_{\gamma(u)} = y_{e_0}^* + \sum_{e \in E_i^+(T_{e_0}^N) \setminus e_0} y_e^* - \sum_{e \in E_i^-(T_{e_0}^N)} y_e^*. \quad (8)$$

Due to the inequality $y_{e_0}^* > x_{e_0}^*$, using (7)–(8), there exists an arc $e_1 \neq e_0$ incident to u such that $y_{e_1}^* > x_{e_1}^*$ if e_1 and e_0 have the same orientation at u (e_1 is ingoing from u and e_0 is outgoing from u), or $y_{e_1}^* < x_{e_1}^*$, otherwise. Similarly, we can find arc $e_{-1} \neq e_0$ incident to v such that $y_{e_{-1}}^* > x_{e_{-1}}^*$ if e_{-1} and e_0 have the same orientation at v , or $y_{e_{-1}}^* < x_{e_{-1}}^*$, otherwise. A similar argument can be applied recursively utilizing the inequalities between value of components of x^*, y^* and the equality constraint (1), (3) in linear programming (I) and (II) on each vertex, respectively. Continuing further all the way down to the leaves of $T_{e_0}^N$, we will finally obtain a path denoted by $P = \{e_{-N}, \dots, e_{-1}, e_0, e_1, \dots, e_N\}$ such that for $-N \leq i \leq N$,

$$y_{e_i}^* > x_{e_i}^* \Leftrightarrow \text{both } e_i \text{ and } e_0 \text{ have the same orientation}, \quad (9)$$

$$y_{e_i}^* < x_{e_i}^* \Leftrightarrow \text{both } e_i \text{ and } e_0 \text{ have the opposite orientation}. \quad (10)$$

According to the definitions of x^* and y^* , such a path is guaranteed to exist. [Fig. 2](#) depicts an example of such a path given by dashed arcs.

Let $\max_E(x) = \max\{w_e x_e : e \in E\}$, where x is a feasible solution of the $\mathcal{WMF}$. If a feasible solution y' of $\mathcal{WMF}_{e_0}^N$ can be obtained by modifying y^* such that

$$\max_{E(T_{e_0}^N)}(y^*) > \max_{E(T_{e_0}^N)}(y'),$$

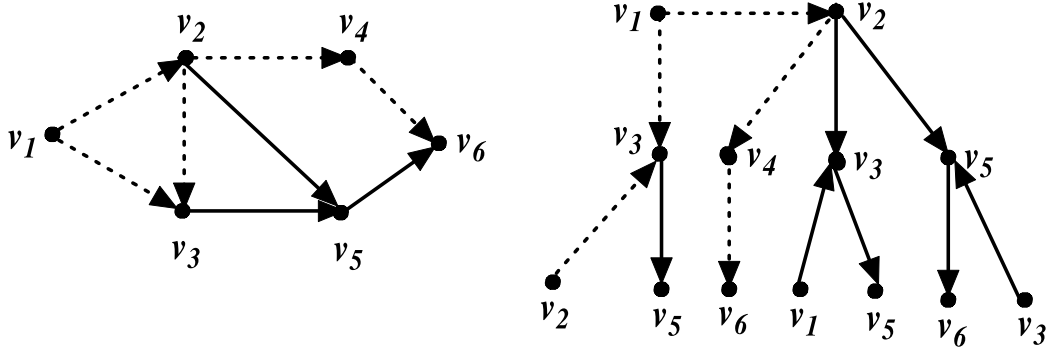


Fig. 2. An example of the path P on a computation tree $T_{v_1 v_2}^2$ with dashed arcs.

then a contradiction arises to the optimality of y^* . Define $\mathcal{A} = \{e \in P : y_e^* > x_e^*\}$ and $\mathcal{B} = \{e \in P : y_e^* < x_e^*\}$. As both $\mathcal{A}$ and $\mathcal{B}$ are finite sets, there exists $\varepsilon > 0$ such that $y_e^* - \varepsilon \geq x_e^*$ for any $e \in \mathcal{A}$ and $y_e^* + \varepsilon \leq x_e^*$ for any $e \in \mathcal{B}$. Let

$$y'_e = \begin{cases} y_e^* - \varepsilon, & e \in \mathcal{A} \\ y_e^* + \varepsilon, & e \in \mathcal{B} \\ y_e^*, & \text{otherwise.} \end{cases}$$

Then $y'_e = y_e^* - \varepsilon \geq x_e^* \geq 0$ for any $e \in \mathcal{A}$ and $y'_e = y_e^* + \varepsilon \leq x_e^* \leq c_e$ for any $e \in \mathcal{B}$, which satisfies all the capacity constraints of (II). Next, we will distinguish three cases below to show that y' satisfies all the other equality constraints of (I). Note that, by the definition of y' , we only need to consider the edges belonging to $E(P) = \mathcal{A} \cup \mathcal{B}$. For any vertex $i \in V^0(T_{e_0}^N)$, let $e', e'' \in \mathcal{A} \cup \mathcal{B}$ be the arcs incident to i .

Case 1. e' and e'' have the same orientation as e_0 .

By (9) and the definition of $\mathcal{A}$, we have that $e', e'' \in \mathcal{A}$. Since e' and e'' are in the same orientation, one of $\{e', e''\}$ is an arc entering i , and the other is an arc leaving i . Without loss of generality, we assume that $e' \in E_i^+(T_{e_0}^N)$ and $e'' \in E_i^-(T_{e_0}^N)$. Then

$$\begin{aligned} & \sum_{e \in E_i^+(T_{e_0}^N)} y'_e - \sum_{e \in E_i^-(T_{e_0}^N)} y'_e \\ &= (y'_{e'} + \sum_{e \in E_i^+(T_{e_0}^N) \setminus e'} y'_e) - (y'_{e''} + \sum_{e \in E_i^-(T_{e_0}^N) \setminus e''} y'_e) \\ &= (y_{e'}^* - \varepsilon + \sum_{e \in E_i^+(T_{e_0}^N) \setminus e'} y_e^*) - (y_{e''}^* - \varepsilon + \sum_{e \in E_i^-(T_{e_0}^N) \setminus e''} y_e^*) \\ &= \sum_{e \in E_i^+(T_{e_0}^N)} y_e^* - \sum_{e \in E_i^-(T_{e_0}^N)} y_e^* \\ &= f_{y'}(i). \end{aligned}$$

Case 2. e' and e'' have the opposite orientation as e_0 .

By (10) and the definition of $\mathcal{B}$, we have that $e', e'' \in \mathcal{B}$. Since e' and e'' are in the same orientation, one of $\{e', e''\}$ is an arc entering i , and the other is an arc leaving i . Without loss of generality, we assume that $e' \in E_i^+(T_{e_0}^N)$ and $e'' \in E_i^-(T_{e_0}^N)$. Then

$$\begin{aligned} & \sum_{e \in E_i^+(T_{e_0}^N)} y'_e - \sum_{e \in E_i^-(T_{e_0}^N)} y'_e \\ &= (y'_{e'} + \sum_{e \in E_i^+(T_{e_0}^N) \setminus e'} y'_e) - (y'_{e''} + \sum_{e \in E_i^-(T_{e_0}^N) \setminus e''} y'_e) \\ &= (y_{e'}^* + \varepsilon + \sum_{e \in E_i^+(T_{e_0}^N) \setminus e'} y_e^*) - (y_{e''}^* + \varepsilon + \sum_{e \in E_i^-(T_{e_0}^N) \setminus e''} y_e^*) \\ &= \sum_{e \in E_i^+(T_{e_0}^N)} y_e^* - \sum_{e \in E_i^-(T_{e_0}^N)} y_e^* \\ &= f_{y'}(i). \end{aligned}$$

Case 3. e' and e'' are in the opposite orientation.

In this case, there is exactly one of $\{e', e''\}$ has the same orientation as e_0 . Without loss of generality, we assume that e' has the same orientation as e_0 , and e'' has the opposite orientation as e_0 . Then by (9) and (10), we have that $e' \in \mathcal{A}$ and $e'' \in \mathcal{B}$. Since e' and e'' are in the opposite orientation, both e' and e'' are arcs entering or leaving i .

- If both e' and e'' are arcs entering i , then

$$\begin{aligned}
 & \sum_{e \in E_i^+(T_{e_0}^N)} y'_e - \sum_{e \in E_i^-(T_{e_0}^N)} y'_e \\
 &= \sum_{e \in E_i^+(T_{e_0}^N)} y'_e - (y'_{e'} + y'_{e''} + \sum_{e \in E_i^-(T_{e_0}^N) \setminus \{e', e''\}} y'_e) \\
 &= \sum_{e \in E_i^+(T_{e_0}^N)} y_e^* - ((y_{e'}^* - \varepsilon) + (y_{e''}^* + \varepsilon) + \sum_{e \in E_i^-(T_{e_0}^N) \setminus \{e', e''\}} y_e^*) \\
 &= \sum_{e \in E_i^+(T_{e_0}^N)} y_e^* - \sum_{e \in E_i^-(T_{e_0}^N)} y_e^* \\
 &= f_Y(i).
 \end{aligned}$$

- If both e' and e'' are arcs leaving i , then

$$\begin{aligned}
 & \sum_{e \in E_i^+(T_{e_0}^N)} y'_e - \sum_{e \in E_i^-(T_{e_0}^N)} y'_e \\
 &= (y'_{e'} + y'_{e''} + \sum_{e \in E_i^+(T_{e_0}^N) \setminus \{e', e''\}} y'_e) - \sum_{e \in E_i^-(T_{e_0}^N)} y'_e \\
 &= ((y_{e'}^* - \varepsilon) + (y_{e''}^* + \varepsilon) + \sum_{e \in E_i^+(T_{e_0}^N) \setminus \{e', e''\}} y_e^*) - \sum_{e \in E_i^-(T_{e_0}^N)} y_e^* \\
 &= \sum_{e \in E_i^+(T_{e_0}^N)} y_e^* - \sum_{e \in E_i^-(T_{e_0}^N)} y_e^* \\
 &= f_Y(i).
 \end{aligned}$$

This implies y' is a feasible solution of $\mathcal{WMMF}_{e_0}^N$.

Now, we only need to show that $\max_{E(T_{e_0}^N)}(y^*) > \max_{E(T_{e_0}^N)}(y')$. Since the only difference between y^* and y' is the flow values of arcs in P , it is sufficient to show that $\max_{E(P)}(y^*) > \max_{E(P)}(y')$. Suppose on the contrary that

$$\max_{E(P)}(y^*) \leq \max_{E(P)}(y'). \quad (11)$$

Note that P can be decomposed into a simple undirected path and some simple undirected cycles in G . Let $\mathcal{C}$ denote the set of these simple undirected cycles. Since the length of the path P after N iterations is $2N + 1$ and thus $\frac{2N+1}{n} \geq \frac{n+1}{n} > 1$, there must exist at least one undirected cycle $C \in \mathcal{C}$. Let

$$x'_e = \begin{cases} x_e^* + \varepsilon, & e \in \mathcal{A} \cap E(C) \\ x_e^* - \varepsilon, & e \in \mathcal{B} \cap E(C) \\ x_e^*, & \text{otherwise.} \end{cases}$$

By a similar argument as y' , it is not difficult to show that x' is a feasible solution of $\mathcal{WMMF}$ on the original graph G . According to the definitions of x' and y' , it follows from (11) that

$$\max_{E(P)}(x') \leq \max_{E(P)}(x^*). \quad (12)$$

Due to (12), we have that

$$\max_E(x^*) \geq \max_{E(P)}(x^*) \geq \max_{E(P)}(x') \geq \max_{E(C)}(x').$$

Since the only difference between x^* and x' is the flow values of arcs in C , by the definitions of x' , it is to see that x' is a feasible solution of $\mathcal{WMMF}$ such that $\max_E(x') \leq \max_E(x^*)$. This leads to a contradiction that x^* is the unique optimal solution of $\mathcal{WMMF}$ which completes the proof. $\square$

5. Conclusion

As a distributed, message-passing algorithm, Belief Propagation (BP) algorithm has been widely used in areas like modern statistics, coding theory, combinatorial optimization and artificial intelligence. Despite empirical successes of BP algorithm in many practical scenarios, the theoretical understanding of the performance of BP algorithm remains far from complete. In this paper, we derive a min–max BP algorithm for the weighted min–max flow ($WMMF$) problem and analyze the correctness and convergence of the algorithm presented. We prove that min–max BP algorithm converges to the optimal solution with fully-polynomial running time, provided that the optimal solution is unique. Moreover, based on the research results and contributions of Gamarnik et al. [11], a simple modification of BP algorithm can be provided to obtain a fully polynomial-time randomized approximation scheme (FPRAS) without requiring the uniqueness of the optimal solution. Finally, it remains open for future research to study more general optimization problems and viability of BP algorithms for them.

Acknowledgments

We are very grateful to the reviewers for their invaluable suggestions and comments, which greatly help to improve the manuscript. The research was partially supported by National Natural Science Foundation of China under Grant Nos. 11871280, U1811461, 61772005, 11971349 and Natural Science Foundation of Guangdong Province of China under Grant No. 2020B1515310009.

References

- [1] D. Achlioptas, F. Ricci-Tersenghi, On the solution-space geometry of random constraint satisfaction problems, in: *Proceedings of the Thirty-Eighth Annual ACM Symposium on Theory of Computing*, ACM, 2006, pp. 130–139.
- [2] M. Bayati, C. Borgs, J. Chayes, R. Zecchina, Belief propagation for weighted b-matchings on arbitrary graphs and its relation to linear programs with integer solutions, *SIAM J. Discrete Math.* 25 (2011) 989–1011.
- [3] M. Bayati, D. Shah, M. Sharma, Max-product for maximum weight matching: convergence, correctness, and LP duality, *IEEE Trans. Inform. Theory* 54 (2008) 1241–1251.
- [4] T. Brunsch, K. Cornelissen, B. Manthey, H. Röglin, Smoothed analysis of belief propagation for minimum-cost flow and matching, in: *WALCOM: Algorithms and Computation*, Springer, Berlin Heidelberg, 2012, pp. 182–193.
- [5] R.E. Burkard, A general hungarian method for the algebraic transportation problem, *Discrete Math.* 22 (1978) 219–232.
- [6] Y. Cheng, M. Neely, K.M. Chugg, Iterative message passing algorithm for bipartite maximum weighted matching, in: *Proceedings of IEEE International Symposium Information Theory*, Cambridge, 2006, pp. 1934–1938.
- [7] G. Dai, F. Li, Y. Sun, D. Xu, X. Zhang, Convergence and correctness of belief propagation for the Chinese postman problem, *J. Global Optim.* 75 (2019) 813–831.
- [8] H.A. Eiselt, M. Gendreau, An optimal algorithm for weighted minimax flow centers on trees, *Transp. Sci.* 25 (1991) 314–316.
- [9] B.J. Frey, D. Dueck, Clustering by passing messages between data points, *Science* 315 (2007) 972–976.
- [10] S. Fujishige, A. Nakayama, W. Cui, On the equivalence of the maximum balanced flow problem and the weighted minimax flow problem, *Oper. Res. Lett.* 5 (1986) 207–209.
- [11] D. Gamarnik, D. Shah, Y. Wei, Belief propagation for min-cost network flow: convergence & correctness, *Oper. Res.* 60 (2012) 410–428.
- [12] T. Ichimori, H. Ishii, T. Nishida, Finding the weighted minimax flow in a polynomial time, *J. Oper. Res. Soc. Japan* 23 (1980) 268–271.
- [13] T. Ichimori, H. Ishii, T. Nishida, Weighted minimax real-valued flows, *J. Oper. Res. Soc. Japan* 24 (1981) 52–59.
- [14] T. Ichimori, M. Murata, H. Ishii, T. Nishida, Minimax cost flow problem, *Technol. Repts. Osaka Univ.* 30 (1980) 39–44.
- [15] M. Mézard, Passing messages between disciplines, *Science* 301 (2003) 1685–1686.
- [16] M. Mézard, G. Parisi, R. Zecchina, Analytic and algorithmic solution of random satisfiability problems, *Science* 297 (2002) 812–815.
- [17] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Reasoning*, Morgan Kaufmann, CA, 1988.
- [18] T. Richardson, R. Urbanke, The capacity of low-density parity check codes under message-passing decoding, *IEEE Trans. Inform. Theory* 47 (2001) 599–618.
- [19] S. Sanghavi, D. Malioutov, A. Willsky, Belief propagation and LP relaxation for weighted matching in general graphs, *IEEE Trans. Inform. Theory* 54 (2011) 2203–2212.
- [20] S. Sanghavi, D. Shah, A.S. Willsky, Message passing for maximum weight independent set, *IEEE Trans. Inform. Theory* 55 (2009) 4822–4834.
- [21] D.F. Stanat, G.A. Magó, Minimizing maximum flows in linear graphs, *Networks* 9 (1979) 333–361.
- [22] Y. Weiss, W. Freeman, On the optimality of solutions of the max-product belief-propagation algorithm in arbitrary graphs, *IEEE Trans. Inform. Theory* 47 (2001) 736–744.
- [23] J. Yedidia, W. Freeman, Y. Weiss, Constructing free-energy approximations and generalized belief propagation algorithms, *IEEE Trans. Inform. Theory* 51 (2005) 2282–2312.